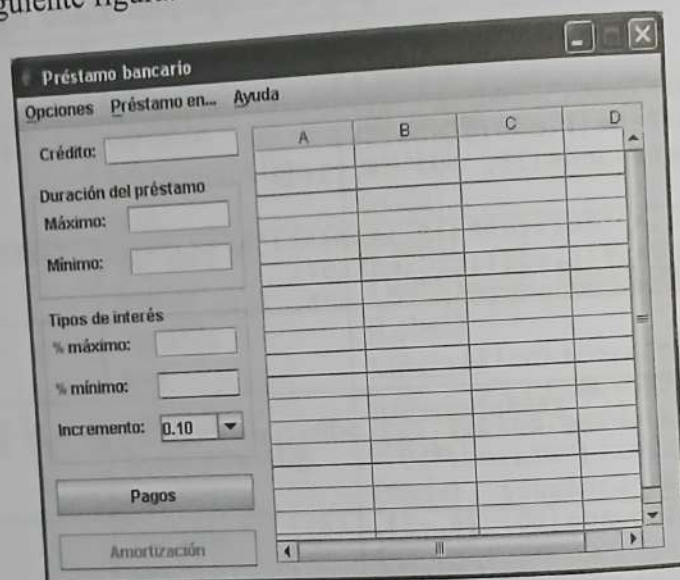


Según vimos en el capítulo 2, la caja de texto *jtfCredito* quedará enfocada cuando se inicie la aplicación por estar colocada la primera.

Sobre el código anterior cabe destacar algunas cosas de interés: una, cómo se añade a la interfaz gráfica un marco con título (fíjese en el marco definido por el borde de *jPanel1*); otra, cómo iniciar una lista desplegable con unos valores determinados (fíjese en el componente *jcbIncremento*); finalmente, observe el código correspondiente al botón *jbtCalculoAmort*; este botón se presenta inicialmente inhabilitado.

Después de hacer las operaciones indicadas, el resultado será similar al presentado en la siguiente figura:



Iniciar la tabla

El diseño inicial que hemos realizado de la ventana *Préstamo bancario* incluye una tabla construida a partir de la clase **JTable**. Esta presenta una cabecera de columnas en la que mostraremos los períodos de amortización, pero no una cabecera de filas, como se puede observar en la figura siguiente, en la que presentar los tipos de interés:

```

jScrollPane.setViewportView(jtablaPrestamo);

// Crear una vista que será utilizada para colocar la tabla
// cabecera de las filas en el mismo panel de desplazamiento
// que la tabla préstamo.
javax.swing.JViewport jv = new javax.swing.JViewport();
jv.setView(jtablaCabsFilas);
jv.setPreferredSize(jtablaCabsFilas.getMaximumSize());
jScrollPane.setRowHeader(jv);
}

```

El modelo de datos de la tabla préstamo define una matriz para almacenar los datos de la tabla, otra para almacenar las cabeceras de las columnas y otra para especificar las columnas que son editables. Así mismo define, entre otros, los métodos `getValueAt` y `setValueAt` para acceder a los datos de una celda individual.

Los modelos de las columnas redefinen el método `addColumn` que es invocado siempre que se añade una columna a la tabla. Precisamente hemos redefinido este método para, en un caso, permitir añadir todas las columnas menos la primera, y en otro, añadir solo la primera columna.

El resto de código está suficientemente comentado como para que se pueda entender sin necesidad de abundar en más explicaciones.

Iniciar la ventana de la aplicación

El siguiente paso es añadir el código necesario para que nuestra aplicación realice las funciones especificadas anteriormente. En primer lugar, vamos a pensar en lo que deseamos que ocurra cuando se inicie la ejecución de la aplicación. Esto puede resumirse en los puntos siguientes:

- Por estética, colocaremos la ventana en el centro de la pantalla utilizando la función miembro `setLocationRelativeTo`. Este método tiene un argumento de tipo `Component` que se toma como referencia para colocar la ventana. Si este argumento es `null`, la ventana se colocará centrada en la pantalla. Para ello, añade al constructor de la clase *Prestamos* el código siguiente:

```
setLocationRelativeTo(null);
```

- La lista desplegable *jcbIncremento* hay que llenarla con los incrementos que estimemos convenientes; por ejemplo, 0.1, 0.25, 0.5 y 1, operación que ya hicimos anteriormente durante el diseño de la interfaz gráfica.
- En el constructor *Prestamo* hemos asignado los valores por omisión, 18 y 4, para el número de filas y de columnas, respectivamente, de la tabla. Por lo

tanto, poder
filas no fija
jas, 4. Esto
del intervalo
lo elegido
dades de ti

- Asumiremos
Esto impli
ya ha sido
ner al mar
- Iniciamos
con el val
- Finalment

Según lo
continuación:

```
public class
```

```
{
    /** Crear
```

```
public Pr
```

```
{
```

```
    setTitl
```

```
    setSize
```

```
    initCom
```

```
    setLoca
```

```
    tiposIn
```

```
    añosMes
```

```
    jmItemA
```

```
    jPanell
```

```
        new j
```

```
        // Dato
```

```
        jtfCred
```

```
        jtfPeri
```

```
        jtfPeri
```

```
        jtfInte
```

```
        jtfInte
```

```
        jcbIncr
```

```
        initTab
```

```
    }
```

```
    // ...
```

```
}
```



```

interesMin = Double.parseDouble(jtfInteresMin.getText());
interesMax = Double.parseDouble(jtfInteresMax.getText());
incremento = Double.parseDouble((String)jcbIncremento.getSelectedItem());

// Comprobar que los datos son válidos
if (credito <= 0 ||
    periodoMin <= 0 || periodoMax <= 0 || periodoMax < periodoMin ||
    interesMin < 0 || interesMax < 0 || interesMax < interesMin)
    throw new NumberFormatException();
}
catch (NumberFormatException e)
{
    javax.swing.JOptionPane.showMessageDialog(
        null, "Datos no válidos",
        "Error", javax.swing.JOptionPane.ERROR_MESSAGE);
    return;
}

// Calcular el nº de tipos de interés y de períodos
tiposIntrs = (int)((interesMax-interesMin)/incremento) + 1;
añosMeses = (periodoMax - periodoMin) + 1;

// Tamaño mínimo de la tabla: los valores iniciales
int filas = tiposIntrs, cols = añosMeses;
if (tiposIntrs < 18) filas = 18;
if (añosMeses < 4) cols = 4;

// Crear la tabla
initTable(filas, cols + 1);

// Almacenar en la columna 0 los tipos de interés
jtablaCabsFilas.setValueAt(AlinDer("##0.00", interesMin)+"%", 0, 0);
for (int fila = 1; fila < tiposIntrs; ++fila)
    jtablaCabsFilas.setValueAt(AlinDer("##0.00",
        interesMin + incremento * fila)+"%", fila, 0);

// Almacenar en la fila 0 las distintas duraciones del préstamo
javax.swing.table.TableColumn colum = null;
String per = " años";
if (jmItemAños.isEnabled()) per = " meses";
for (int columna = 0; columna < añosMeses; ++columna)
{
    colum = jtablaPrestamo.getColumnModel().getColumn(columna);
    colum.setHeaderValue((periodoMin + columna) + per);
}

// Los períodos ¿en qué vienen dados? ¿En años o en meses?
int P = 0;
if (!jmItemAños.isEnabled())
    P = 12; // son años
else
    P = 1; // son meses

// Calcular pagos

```

```
// Borrar el nodo seleccionado
modeloÁrbol.removeNodeFromParent(nodoSelec);
}
```

Borrar todos los nodos

Esta orden la ejecutará el usuario cuando quiera borrar todos los nodos del árbol, excepto la raíz. Este evento de acción genera el mensaje `actionPerformed`, al que la aplicación responderá ejecutando el método `jpmlItemBorrarArbolActionPerformed` indicado a continuación, el cual invocará al método `removeAllChildren` de `DefaultMutableTreeNode` para borrar todos los nodos hijos de la raíz del árbol y después invocará a `reload` de `DefaultTreeModel` para actualizar la vista del árbol (ahora mostrará solo el nodo raíz).

```
private void jpmlItemBorrarArbolActionPerformed(java.awt.event.ActionEvent evt)
{
    raíz.removeAllChildren(); // borrar todos los nodos
    modeloÁrbol.reload();      // refrescar la vista
}
```

Personalizar el aspecto de un árbol

La figura del árbol *Teléfonos* mostrada unas páginas atrás visualiza por cada nodo un icono predefinido y un texto. En general, el aspecto que muestra el árbol depende del estilo de interfaz seleccionado: *Java*, *Windows* o *Motif X-Window*. Este aspecto puede ser modificado dentro de unos límites. Algunas de las operaciones que podemos realizar para modificar el aspecto del árbol son las siguientes:

- Mostrar o no el botón que permite expandir o recoger el nodo raíz. Esto se hace invocando al método `setShowsRootHandles` con el argumento `true`:

```
jTree1.setShowsRootHandles(true);
```

- Cambiar los iconos utilizados por omisión por los nodos hoja y por los nodos de bifurcación recogidos y expandidos. Para ello, hay que saber cómo pintar un árbol sus nodos. Pues utilizando un objeto de la clase `DefaultTreeCellRenderer`. Por lo tanto, primero hay que crear este objeto "pintor", y después, utilizando sus métodos `setLeafIcon`, `setClosedIcon` y `setOpenIcon`, le indicaremos los iconos que tiene que utilizar para pintar los nodos hoja y los nodos de bifurcación recogidos y expandidos, respectivamente (un valor `null` no mostraría el icono). Una vez realizadas estas operaciones, asignaremos ese "pintor" al árbol, invocando a su método `setCellRenderer`. Para practicar lo expuesto en la aplicación que venimos desarrollando, añade el código siguiente en `initComponents`:

```
javax.swing.tree.DefaultTreeCellRenderer pintor =
```


La figura siguiente muestra el aspecto final de la aplicación. Observe, además de lo expuesto anteriormente, otros detalles, como una lista desplegable para tipos de interés y las barras de desplazamiento.

	29 años	30 años
2,00%	682,08	665,32
2,50%	727,72	711,22
3,00%	775,97	758,89
3,50%	824,10	808,28
4,00%	874,75	859,35
4,50%	926,99	912,03
5,00%	980,75	966,28
5,50%	1.035,98	1.022,02
6,00%	1.092,61	1.079,15
6,50%	1.150,58	1.137,72
7,00%	1.209,83	1.197,64
7,50%	1.270,30	1.258,69
8,00%	1.331,90	1.320,78

Para empezar, cree una nueva aplicación denominada *Préstamo* que utilice un formulario de la clase **JFrame** como ventana principal.

Asigne a la ventana el título *Préstamo bancario*, un tamaño de 485×385 y ponga su propiedad *resizable* (**setResizable**) a valor **false** para que no se pueda redimensionar la ventana.

A continuación, añada los menús *Opciones* y *Préstamo en...* con las órdenes que se especifican en la tabla siguiente:

Objeto	Propiedad	Valor
Opciones	variable	jmnuOpciones
	text	Opciones
	mnemonic	'O'
Instrucciones	variable	jmItemInstruc
	text	Instrucciones...
	mnemonic	'I'
Separador	variable	jSeparador1
Salir	variable	jmItemSalir
	text	Salir
	mnemonic	'S'
Préstamo en...	variable	jmnuPrestanoEn
	text	Préstamo en...
	mnemonic	'P'

Años

Meses

Ayuda

Acerca de

El código
en la tabla anterior

```
/*
 * Prestamo
 */
package Cap
```

```
/**
 *
 * @author
 */
public clas
```

```
{
    /** Crea
    public P
    {
        setTit
        setSiz
        initCo
    }
```

```
private
{
```

```
    jmbaB
    jmnuOp
    jmItem
    jSepar
    jmItem
    jmnuPr
    jmItem
    jmItem
    jmnuAy
    jmItem
```

```
getCor
```

```
setRes
```

```

        System.exit(0);
    }

    /** Salir de la aplicación */
    private void exitForm(java.awt.event.WindowEvent evt)
    {
        System.exit(0);
    }

    public static void main(String args[])
    {
        new Prestamo().setVisible(true);
    }

    // Declaración de variables
    private javax.swing.JMenuBar jmbarBarraDeMenus;
    private javax.swing.JMenu jmnuOpciones;
    private javax.swing.JMenuItem jmItemInstruc;
    private javax.swing.JSeparator jSeparador1;
    private javax.swing.JMenuItem jmItemSalir;
    private javax.swing.JMenu jmnuPrestamoEn;
    private javax.swing.JMenuItem jmItemAños;
    private javax.swing.JMenuItem jmItemMeses;
    private javax.swing.JMenu jmnuAyuda;
    private javax.swing.JMenuItem jmItemAcercaDe;
}

```

A continuación, añade un panel de desplazamiento para la rejilla:

```

private javax.swing.JScrollPane jScrollPane1;
// ...
jScrollPane1 = new javax.swing.JScrollPane();
jScrollPane1.setBounds(180, 10, 290, 310);
getContentPane().add(jScrollPane1);

```

Ahora, siguiendo los pasos estudiados al principio de este capítulo, añade una tabla y haga que sea la vista del panel de desplazamiento *jScrollPane1*. Para mayor claridad, realice estas operaciones desde un método privado *initTable* de la clase *Prestamo*:

```

private javax.swing.JTable tablaPrestamo;
// ...
private void initTable(final int filasTabla, final int colsTabla)
{
    tablaPrestamo = new javax.swing.JTable(filasTabla, colsTabla);
    tablaPrestamo.setAutoResizeMode(javax.swing.JTable.AUTO_RESIZE_OFF);
    jScrollPane1.setViewportView(tablaPrestamo);
}

```

El método **setAutoResizeMode** de **JTable** permite especificar cómo se redimensionarán de forma automática las columnas de la tabla. Cuando el argumento especificado es **AUTO_RESIZE_OFF**, el redimensionamiento no ocurrirá de


```

        super.addColumn(col);
    }

    // Crear un modelo de columnas para la tabla que hará de cabecera
    // de las filas, que solo tenga en cuenta la primera columna.
    javax.swing.table.TableColumnModel modeloCabsFilas =
        new javax.swing.table.DefaultTableColumnModel()
    {
        boolean primeraCol = true;
        public void addColumn(javax.swing.table.TableColumn col)
        {
            if (primeraCol)
            {
                col.setMaxWidth(55);
                super.addColumn(col);
                primeraCol = false;
            }
            // Ignorar el resto de las columnas
        }
    };

    // Crear el modelo de la tabla préstamo
    javax.swing.table.TableModel modeloTabla = new CModeloTablaPrestamo();

    // Crear la tabla préstamo con los modelos pasados como argumentos.
    jTableaPrestamo = new javax.swing.JTable(modeloTabla, modeloColumns);

    // Crear la tabla cabecera de las filas.
    jTableaCabsFilas = new javax.swing.JTable(modeloTabla, modeloCabsFilas);

    // Crear las columnas (createDefaultColumnsFromModel utiliza
    // el método getColumnCount definido en TableModel).
    jTableaPrestamo.createDefaultColumnsFromModel();
    jTableaCabsFilas.createDefaultColumnsFromModel();

    // Asegurar el sincronismo entre las dos tablas cuando se
    // realicen selecciones, haciendo que compartan el mismo modelo.
    jTableaPrestamo.setSelectionModel(jTableaCabsFilas.getSelectionModel());

    // Color gris para la cabecera de las filas y hacer la selección
    // sobre la misma invisible.
    jTableaCabsFilas.setBackground(java.awt.Color.lightGray);
    jTableaCabsFilas.setSelectionBackground(java.awt.Color.lightGray);

    // Permitir barras de desplazamiento para ambas tablas
    jTableaPrestamo.setAutoResizeMode(javax.swing.JTable.AUTO_RESIZE_OFF);
    jTableaCabsFilas.setAutoResizeMode(javax.swing.JTable.AUTO_RESIZE_OFF);

    // Establecer la fuente Courier New
    jTableaPrestamo.setFont(new java.awt.Font("Courier New", 0, 12));
    jTableaCabsFilas.setFont(new java.awt.Font("Courier New", 0, 12));

    // Incluir la tabla préstamo en un panel de desplazamiento

```

puesta a este evento, se ejecuta el método *jtablaPrestamoMouseClicked*, que asociaremos con la tabla así:

```
jtablaPrestamo.addMouseListener(new java.awt.event.MouseAdapter()
{
    public void mouseClicked(java.awt.event.MouseEvent evt)
    {
        jtablaPrestamoMouseClicked(evt);
    }
});
```

El método *jtablaPrestamoMouseClicked* obtendrá el dato de la celda sobre la que el usuario hizo clic, lo almacenará en la variable *pagoMensual*, y activará el botón *Amortización* siempre que la celda sea una celda válida; esto es, que contenga una cantidad, y la tabla visualizada sea la de pagos y no la de amortización. Añada este método a la clase *Prestamo* tal y como se muestra a continuación:

```
private void jtablaPrestamoMouseClicked(java.awt.event.MouseEvent evt)
{
    Object datoCelda = jtablaPrestamo.getValueAt(
        jtablaPrestamo.getSelectedRow(),
        jtablaPrestamo.getSelectedColumn());
    if (datoCelda != null && tablaPagos)
    {
        // Pasar el número a formato de US necesario para parseDouble
        StringBuffer s = new StringBuffer(datoCelda.toString());
        for (int i = 0; i < s.length(); ++i)
        {
            if (s.charAt(i) == '.') s.delete(i, i+1);
            if (s.charAt(i) == ',') s.setCharAt(i, '.');
        }
        // Convertir a double
        pagoMensual = Double.parseDouble(s.toString());
        jbtCalculoAmort.setEnabled(true);
    }
}
```

Añada también la variable *pagoMensual* de tipo *double* a la clase *Prestamo* como propiedad privada de la misma.

Para visualizar la tabla de amortización a la que nos hemos referido anteriormente, el usuario hará clic en el botón *Amortización*. Como respuesta, se ejecutará el método *jbtCalculoAmortActionPerformed*, que asociaremos con el botón así:

```
jbtCalculoAmort.addActionListener(new java.awt.event.ActionListener()
{
    public void actionPerformed(java.awt.event.ActionEvent evt)
    {
        jbtCalculoAmortActionPerformed(evt);
    }
});
```



```

addWindowListener(new java.awt.event.WindowAdapter()
{
    public void windowClosing(java.awt.event.WindowEvent evt)
    {
        exitForm(evt);
    }
});

jmenuOpciones.setMnemonic('O');
jmenuOpciones.setText("Opciones");
jmenuItemInstruc.setMnemonic('I');
jmenuItemInstruc.setText("Instrucciones...");
jmenuOpciones.add(jmenuItemInstruc);

jmenuOpciones.add(jseparator1);

jmenuItemSalir.setMnemonic('S');
jmenuItemSalir.setText("Salir");
jmenuItemSalir.addActionListener(new java.awt.event.ActionListener()
{
    public void actionPerformed(java.awt.event.ActionEvent evt)
    {
        jMenuItemSalirActionPerformed(evt);
    }
});

jmenuOpciones.add(jmenuItemSalir);

jmenuBarraDeMenus.add(jmenuOpciones);

jmenuPrestamoEn.setMnemonic('P');
jmenuPrestamoEn.setText("Préstamo en...");
jmenuItemAños.setMnemonic('A');
jmenuItemAños.setText("Años");
jmenuPrestamoEn.add(jmenuItemAños);

jmenuItemMeses.setMnemonic('M');
jmenuItemMeses.setText("Meses");
jmenuPrestamoEn.add(jmenuItemMeses);
jmenuBarraDeMenus.add(jmenuPrestamoEn);

jmenuAyuda.setMnemonic('A');
jmenuAyuda.setText("Ayuda");
jmenuItemAcercaDe.setMnemonic('A');
jmenuItemAcercaDe.setText("Acerca de Préstamo...");
jmenuAyuda.add(jmenuItemAcercaDe);

jmenuBarraDeMenus.add(jmenuAyuda);

setJMenuBar(jmenuBarraDeMenus);
}

private void jMenuItemSalirActionPerformed(java.awt.event.ActionEvent evt)
{

```

```

system
}
/** Salir
private
{
    system
}

public
{
    new P

// Decl
private
private
private
private
private
private
private
private
private
private
}

```

A con

```

private
// ...
jScrollPane
jScrollPane
getConte

```

Ahora
tabla y ha
yor clarid
clase Pres

```

private
// ...
private
{
    tablaP
    tablaP
    jScrollPane
}

```

El mé
mensionar
especifica

forma automática, pero podremos utilizar barras de desplazamiento si incluimos la tabla en un panel de desplazamiento.

Declare las propiedades *tiposIntrs* y *añosMeses* e inícielas en el constructor *Prestamo* de la clase aplicación con los valores 18 y 4, respectivamente. Las utilizaremos como argumento cuando invoquemos a *initTable*.

Finalmente, añada el resto de los controles especificados en la tabla que se expone a continuación (el código completo lo encontrará en la carpeta *cap05\Resueltos\Prestamo* del CD que acompaña al libro):

Objeto	Propiedad	Valor
Etiqueta 1	variable text	jLabel1 Crédito:
Caja de texto 1	variable horizontalAlig. text	jtfCredito RIGHT (nada)
Panel 1	variable border	jPanel1 TitledBorder: Duración del préstamo
Etiqueta 2	variable text	jLabel2 Máximo:
Caja de texto 2	variable horizontalAlig. text	jtfPeriodoMax RIGHT (nada)
Etiqueta 3	variable text	jLabel3 Mínimo:
Caja de texto 3	variable horizontalAlig. text	jtfPeriodoMin RIGHT (nada)
Panel 2	variable text	jPanel2 TitledBorder: Tipo de interés
Etiqueta 4	variable text	jLabel4 % máximo:
Caja de texto 4	variable horizontalAlig. text	jtfInteresMax RIGHT (nada)
Etiqueta 5	variable text	jLabel5 % mínimo:
Caja de texto 5	variable horizontalAlig. text	jtfInteresMin RIGHT (nada)

Etiqueta 6

Lista desple

Botón de pu

Botón de p

Suponie
ción se mue
nentes:

```
jLabel1.se  
getContent  
jLabel1.se
```

```
jtfCredito  
getContent  
jtfCredito
```

```
jPanel1.se  
jPanel1.se  
new java  
jLabel2.se  
jPanel1.a  
jLabel2.se
```

```
jtfPeriodo  
jPanel1.a  
jtfPeriodo  
// ...
```

```
jcbIncrem  
jcbIncrem  
new Str  
jPanel2.a  
jcbIncrem
```

```
getConten  
jPanel2.s
```

```
jbtCalcul  
getConten  
jbtCalcul
```

```
jbtCalcul  
jbtCalcul  
getConten  
jbtCalcul
```


tanto, podemos definir con la variable *tiposIntrs* el valor, por omisión, de las filas no fijas, 18, y con *añosMeses* el valor por omisión de las columnas no fijas, 4. Estos valores variarán durante la ejecución de la aplicación en función del intervalo y del incremento elegidos para los tipos de interés, y del intervalo elegido para la duración del préstamo. Defina estas variables como propiedades de tipo **int** de la clase *Prestamo*.

- Asumiremos, por omisión, que la duración del préstamo viene dada en años. Esto implica dejar inhabilitada la orden *Años* del menú *Préstamo en...* (porque ya ha sido elegida), habilitar la orden *Meses* (para que se pueda elegir) y poner al marco "Duración del préstamo" el título *Años del préstamo*.
- Iniciamos las cajas de texto con unos datos simbólicos y la lista desplegable con el valor 0.5 (elemento de índice 2).
- Finalmente, invocamos al método *initTable* para construir la tabla.

Según lo expuesto, edite el constructor de la clase *Prestamo* como se indica a continuación:

```
public class Prestamo extends javax.swing.JFrame
{
    /** Crear un nuevo formulario Prestamo */
    public Prestamo()
    {
        setTitle("Préstamo bancario");
        setSize(455, 385);
        initComponents();

        setLocationRelativeTo(null); // centrar la ventana
        tiposIntrs = 18; // número de filas no fijas
        añosMeses = 4; // número de columnas no fijas
        jMenuItemAños.setEnabled(false);
        jPanel1.setBorder(
            new javax.swing.border.TitledBorder("Años del préstamo"));
        // Datos iniciales
        jTextFieldCredito.setText("6000");
        jTextFieldPeriodoMax.setText("1");
        jTextFieldPeriodoMin.setText("1");
        jTextFieldInteresMax.setText("10.00");
        jTextFieldInteresMin.setText("0.00");
        jComboBoxIncremento.setSelectedIndex(2); // incremento

        initTable(tiposIntrs, añosMeses + 1);
    }
    // ...
}
```


Observe que el segundo argumento pasado por *intTable* es *añosMeses+1*, es, el número de columnas no fijas más una fija.

Manejo de la aplicación

Una vez que tengamos visualizada la ventana, quizás el usuario necesite instrucciones acerca del manejo de la aplicación. Esta ayuda la implementaremos a través de la orden *Instrucciones* del menú *Opciones*, de forma que cuando el usuario ejecute dicha orden, se visualice una ventana con los pasos a seguir. Para ello añadida en el método *initComponents* de *Prestamo* un manejador de eventos de la clase **ActionListener** que vincule el objeto *jmItemInstruc* con el método que responda a la acción "clic" realizada sobre él:

```
jmItemInstruc.addActionListener(new java.awt.event.ActionListener()
{
    public void actionPerformed(java.awt.event.ActionEvent evt)
    {
        jmItemInstrucActionPerformed(evt);
    }
});
```

A continuación añadida a la clase *Prestamo* el método *jmItemInstrucActionPerformed* que mostrará un diálogo de la clase **JOptionPane** con el mensaje especificado a continuación:

```
private void jmItemInstrucActionPerformed(java.awt.event.ActionEvent evt)
{
    String mensaje;
    mensaje = "Introduzca el crédito, la duración del préstamo y el tipo\n";
    mensaje += "de interés. Pulse el botón [Pagos] para visualizar\n";
    mensaje += "los pagos mensuales en la rejilla.\n\n";
    mensaje += "Elija un pago mensual y pulse el botón [Amortización]\n";
    mensaje += "para visualizar el plan de amortización para el interés\n";
    mensaje += "y períodos correspondientes al pago elegido.\n\n";
    mensaje += "Para copiar datos en el portapapeles, seleccione las celdas\n";
    mensaje += "que desee y pulse las teclas Ctrl+c.\n";
    javax.swing.JOptionPane.showMessageDialog(
        null, mensaje, "Instrucciones",
        javax.swing.JOptionPane.INFORMATION_MESSAGE);
}
```

El siguiente paso es introducir los datos *crédito*, *duración del préstamo* y *tipos de interés*. Ahora bien, la duración del préstamo son ¿meses o años? Este concepto puede definirlo el usuario a través de las órdenes *Años* y *Meses* del menú *Préstamo* en... El método asociado con estas órdenes inhabilitará la duración *Años* o *Meses* elegida, habilitará la no elegida para que pueda elegirse la próxima vez y pondrá en el marco que titulamos *Duración del préstamo* un nuevo título

Años del préstamo
diendo de form
el método *initCo*
tionListener qu
acción "clic" re
sos, la respuesta
presentado a co

```
private void jm
{
    Object item
    String titu
    if (item ==
    {
        jmItemAño
        jmItemMes
        tituloMa
    }
    else if (it
    {
        jmItemAño
        jmItemMes
        tituloMa
    }
    jPanell.se
    new java
}
```

La caja de
la variable *cre*

Las cajas
la duración má
periodoMax y

Las cajas
tipo de interés
bles *interesMa*

Después d
ta desplegable
to de tipo *dou*
tener los tipos

Defina las
mo según se in

Años del préstamo o *Meses del préstamo*, acorde con la elección realizada. Procediendo de forma análoga a como lo hicimos con la orden *Instrucciones*, añada en el método *initComponents* de *Prestamo* un manejador de eventos de la clase *ActionListener* que vincule el objeto *jmItemAños* con el método que responderá a la acción "clic" realizada sobre él. Ídem para el objeto *jmItemMeses*. En ambos casos, la respuesta será la ejecución del método *jmItemAñosMesesActionPerformed* presentado a continuación:

```
private void jmItemAñosMesesActionPerformed(java.awt.event.ActionEvent evt)
{
    Object item = evt.getSource();
    String tituloMarco = "";

    if (item == jmItemAños)
    {
        jmItemAños.setEnabled(false);
        jmItemMeses.setEnabled(true);
        tituloMarco = "Años del préstamo";
    }
    else if (item == jmItemMeses)
    {
        jmItemAños.setEnabled(true);
        jmItemMeses.setEnabled(false);
        tituloMarco = "Meses del préstamo";
    }

    jPanell.setBorder(
        new javax.swing.border.TitledBorder(tituloMarco));
}
```

La caja de texto *jtfCredito* recoge la cantidad prestada que almacenaremos en la variable *credito* de tipo **double**.

Las cajas de texto *jtfPeriodoMax* y *jtfPeriodoMin* recogen, respectivamente, la duración máxima y mínima del préstamo, que almacenaremos en las variables *periodoMax* y *periodoMin* de tipo **int**.

Las cajas de texto *jtfInteresMax* e *jtfInteresMin* recogen, respectivamente, el tipo de interés máximo y mínimo del préstamo, que almacenaremos en las variables *interesMax* e *interesMin* de tipo **double**.

Después de los datos anteriores, el usuario seleccionará un elemento de la lista desplegable *jcbIncremento*, cuyo valor almacenaremos en la variable *incremento* de tipo **double**. Este valor se corresponde con el incremento a aplicar para obtener los tipos de interés entre el mínimo y el máximo especificados.

Defina las variables anteriores como propiedades privadas de la clase *Prestamo* según se indica a continuación:

```
private double credito;
private int periodoMax, periodoMin;
private double interesMin, interesMax;
private double incremento;
```

Una vez introducidos todos los datos, el usuario hará clic en el botón *Pagos* (el botón *Amortización*, lógicamente, está inhabilitado). Como respuesta, se ejecutará el método *jbtCalculoPagosActionPerformed*, que asociaremos con este botón:

```
jbtCalculoPagos.addActionListener(new java.awt.event.ActionListener()
{
    public void actionPerformed(java.awt.event.ActionEvent evt)
    {
        jbtCalculoPagosActionPerformed(evt);
    }
});
```

El método *jbtCalculoPagosActionPerformed* tiene como finalidad:

- Obtener de los controles los datos introducidos y verificarlos.
- Calcular el número de tipos de interés y de períodos a partir de los datos introducidos.
- Crear la tabla con un número de filas igual al número de tipos de interés y con un número de columnas igual al número de períodos más la columna fija. Los valores mínimos de filas y de columnas de la tabla serán siempre los valores de los que partimos inicialmente.
- Mostrar en la columna cero, la fija, los tipos de interés.
- Mostrar en la fila cero las distintas duraciones del préstamo.
- Calcular los pagos mensuales para cada período y tipo de interés reflejados en la tabla.
- Mostrar los pagos calculados redondeados a dos cifras decimales y ajustados a la derecha.
- Finalmente, para indicar que la tabla mostrada es la de pagos y no la de amortización, pondrá a **true** la variable *tablaPagos*. Añada esta variable a la clase *Prestamo* como propiedad privada de la misma.

```
private void jbtCalculoPagosActionPerformed(java.awt.event.ActionEvent evt)
{
    // Actualizar las variables con los valores de los controles
    try
    {
        credito = Double.parseDouble(jtfCredito.getText());
        periodoMin = Integer.parseInt(jtfPeriodoMin.getText());
        periodoMax = Integer.parseInt(jtfPeriodoMax.getText());
```

```
interesMi
interesMa
incremento
```

```
// Compr
if (cred
    perio
    inter
    throw r
```

```
}
catch (Numb
{
    javax.sw
```

```
return;
}
```

```
// Calcula
tiposIntrs
añosMeses
```

```
// Tamaño
int filas
if (tiposI
if (añosMe
```

```
// Crear l
initTable (
```

```
// Almacen
jtablaCabs
for (int f
    jtablaCa
    inter
```

```
// Almacen
javax.swir
String per
if (jmItem
for (int c
{
    colum =
    colum.se
}
```

```
// Los per
int P = 0;
if (!jmIte
    P = 12;
else
    P = 1;
```

```
// Calcula
```


El método *jbtCalculoAmortActionPerformed* tiene como finalidad:

- Obtener el tipo de interés y la duración del préstamo correspondiente a la celda seleccionada.
- Crear la tabla con un número de filas igual al número de mensualidades a pagar, y con un número de columnas igual a cinco (una fija y cuatro no fijas). El valor mínimo de las filas de la tabla será siempre el valor del que partimos inicialmente.
- Mostrar en la columna cero los meses (1, 2, 3...) y en la fila cero las cabece-
ras: *Capital, Intereses, Capital pendiente y Total intereses*.
- Calcular y mostrar los pagos mes a mes, hasta la finalización del período del préstamo, desglosados en capital amortizado e intereses pagados. Esto es, la tabla de amortización incluirá, por cada mensualidad, su desglose en capital e intereses, el capital pendiente después de realizar ese pago y el total de los intereses pagados hasta ese momento.
- Finalmente, inhabilitará de nuevo el botón *Amortización* y pondrá la variable *tablaPagos* a valor **false**.

```
private void jbtCalculoAmortActionPerformed(java.awt.event.ActionEvent evt)
{
    // Obtener el tipo de interés correspondiente a la celda seleccionada
    int fila = jtablaPrestamo.getSelectedRow();
    int columna = jtablaPrestamo.getSelectedColumn();
    String sinteres = (String)jtablaCabsFilas.getValueAt(fila, 0);
    sinteres = sinteres.substring(0, sinteres.indexOf('%'));
    sinteres = sinteres.replace(',', '.');
    double interes = Double.parseDouble(sinteres) / 100 / 12;

    // Obtener el período correspondiente a la celda seleccionada
    int P = 0;
    if (!jItemAños.isEnabled())
        P = 12; // son años
    else
        P = 1; // son meses

    javax.swing.table.TableColumn colum = null;
    colum = jtablaPrestamo.getColumnModel().getColumn(columna);
    String smeses = (String)colum.getHeaderValue();
    smeses = smeses.substring(0, smeses.indexOf(' '));
    int meses = Integer.parseInt(smeses) * P;
    int filas = meses, cols = 5;
    if (filas < 18) filas = 18;

    // Crear la tabla
    initTable(filas, cols);

    // Almacenar/mostrar en la columna 0 los meses
    for (int mes = 0; mes < meses; ++mes)
        jtablaCabsFilas.setValueAt(AlinDer("#####", mes+1), mes, 0);
}
```

```
// Almacena
String cab[
for (column
{
    colum = j
    colum.set
}
```

```
// Calcula
// La colum
// pendiente
double inte
double cap
String form
for (int m
```

```
{
    // Cálcu
    interese
    // Cálcu
    capitalM
    // Cálcu
    creditoP
    // Cálcu
    totalInt
    // Capit
    jtablaPre
    // Inter
    jtablaPre
    // Capit
    jtablaPre
    // Total
    jtablaPre
}
jbtCalculo
tablaPagos
}
```

Finalmen
orden visualiz

Para ello,
eventos de la
método que re

Etiqueta 6	variable text	jLabel6 Incremento:
Lista desplegable	variable editable	jcbIncremento true
Botón de pulsación	variable text	jbtCalculoPagos Pagos
Botón de pulsación	variable text enabled	jbtCalculoAmort Amortización false

Suponiendo definidas las variables indicadas en la tabla anterior, a continuación se muestra, a modo de ejemplo, el código para añadir algunos de los componentes:

```

jLabel1.setText("Crédito:");
getContentPane().add(jLabel1);
jLabel1.setBounds(15, 10, 50, 16);

jtfCredito.setHorizontalAlignment(javax.swing.JTextField.RIGHT);
getContentPane().add(jtfCredito);
jtfCredito.setBounds(65, 10, 105, 20);

jPanel1.setLayout(null);
jPanel1.setBorder(
    new javax.swing.border.TitledBorder("Duración del préstamo"));
jLabel2.setText("Máximo:");
jPanel1.add(jLabel2);
jLabel2.setBounds(10, 20, 55, 16);

jtfPeriodoMax.setHorizontalAlignment(javax.swing.JTextField.RIGHT);
jPanel1.add(jtfPeriodoMax);
jtfPeriodoMax.setBounds(75, 20, 80, 20);
// ...

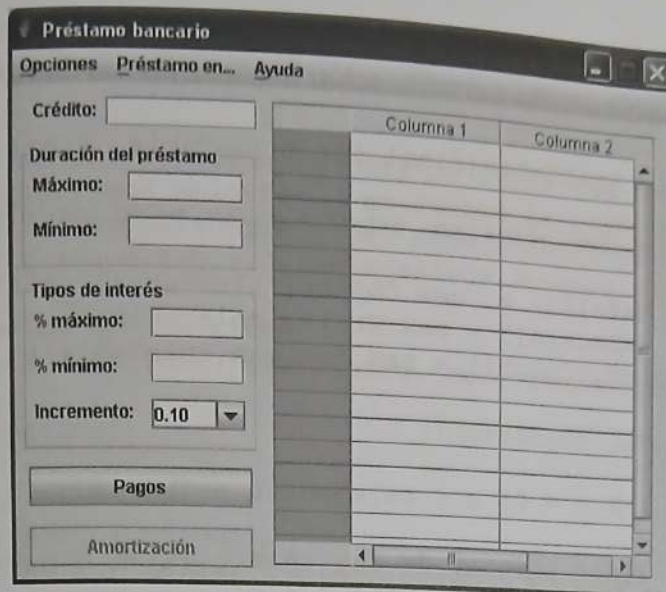
jcbIncremento.setEditable(true);
jcbIncremento.setModel(new javax.swing.DefaultComboBoxModel(
    new String[] { "0.10", "0.25", "0.50", "1.00" }));
jPanel2.add(jcbIncremento);
jcbIncremento.setBounds(90, 80, 65, 20);

getContentPane().add(jPanel2);
jPanel2.setBounds(5, 130, 165, 115);

jbtCalculoPagos.setText("Pagos");
getContentPane().add(jbtCalculoPagos);
jbtCalculoPagos.setBounds(10, 255, 155, 26);

jbtCalculoAmort.setText("Amortización");
jbtCalculoAmort.setEnabled(false);
getContentPane().add(jbtCalculoAmort);
jbtCalculoAmort.setBounds(10, 295, 155, 26);

```

Para realizar el diseño anterior, hay que crear dos tablas, una de una columna (la columna gris) y otra de $n-1$ columnas sincronizadas entre sí para que el desplazamiento vertical se realice como si se tratara de una sola tabla. En cuanto al desplazamiento horizontal, solo será aplicable a la tabla de $n-1$ columnas, la tabla de una columna quedará fija.

Para entender cómo se realizará lo expuesto, recuerde que una tabla mantiene todos sus datos por medio de un objeto (modelo) que implementa la interfaz **TableModel**, y no hay ninguna razón para pensar que dos tablas no puedan compartir el mismo modelo de datos. De esta forma, la tabla que implementa la columna fija (tabla cabecera de las filas) utilizaría la primera columna del modelo; y la que implementa las columnas no fijas, el resto de las columnas del modelo.

El modelo al que nos acabamos de referir lo hemos denominado *modeloTabla* y es de la clase *CModeloTablaPrestamo* derivada de **AbstractTableModel**.

Las columnas son añadidas, borradas o movidas a través de un objeto (modelo) que implemente la interfaz **TableColumnModel**. Entonces, una forma de construir tablas que muestren parte de los datos de un único modelo de datos es construir modelos para las columnas de las tablas que hagan lo que nosotros queramos, como añadir una columna en particular.

En nuestro caso hemos construido dos modelos de columnas: *modeloCabsFijas* para la tabla de una columna y *modeloColums* para la tabla de $n-1$ columnas.

Una vez que tenemos los modelos, es relativamente simple crear dos tablas que utilicen el mismo modelo de datos, pero diferentes modelos para gestionar las columnas. Cada tabla mostrará solamente las columnas que su modelo permite. Después colocaremos una a continuación de la otra. Una actuará como cabecera

de las filas y la otra mostrará los datos, en nuestro caso, acerca del préstamo. El método *initTable* escrito a continuación implementa las dos tablas descritas:

```
private void initTable(final int filasTabla, final int colsTabla)
{
    class CModeloTablaPrestamo extends javax.swing.table.AbstractTableModel
    {
        Object dato[][] = new Object[filasTabla][colsTabla];
        String cabecera[] = new String[colsTabla];
        boolean editColum[] = new boolean[colsTabla];

        CModeloTablaPrestamo()
        {
            for (int c = 0; c < colsTabla; ++c)
            {
                // Texto inicial de las cabeceras de las columnas.
                cabecera[c] = "Columna " + c;
                // Hacer editables las columnas 1 a colsTabla-1
                if (c != 0) editColum[c] = true;
            }
        }

        public int getColumnCount() { return cabecera.length; }
        public int getRowCount() { return dato.length; }
        public String getColumnName(int col) { return cabecera[col]; }

        public Object getValueAt(int fila, int col)
        {
            return dato[fila][col];
        }

        public void setValueAt(Object obj, int fila, int col)
        {
            dato[fila][col] = obj;
        }

        public boolean isCellEditable(int indFila, int indColum)
        {
            return editColum[indColum];
        }
    };

    // Crear un modelo de columnas para la tabla préstamo que ignore
    // la primera columna.
    javax.swing.table.TableColumnModel modeloColumns =
        new javax.swing.table.DefaultTableColumnModel()
    {
        boolean primeraCol = true;

        public void addColumn(javax.swing.table.TableColumn col)
        {
            // Ignorar la primera columna.
            if (primeraCol) { primeraCol = false; return; }
            col.setMinWidth(110);
        }
    };
}
```

```
super.a
}
}
// Crear un
// de las
javax.swing
new jav
{
    boolean
    public vo
    {
        if (pr
        {
            col.
            supe
            prim
        }
        // Ignor
    }
}
// Crear e
javax.swing
// Crear la
jtablaPres
// Crear l
jtablaCabsi
// Crear
// el méto
jtablaPres
jtablaCab
// Asegur
// realic
jtablaPres
// Color
// sobre
jtablaCab
jtablaCab
// Permit
jtablaPres
jtablaCabs
// Establ
jtablaPre
jtablaCab
// Inclui
```



```

double interes = 0.0, pagoMensual = 0.0;
int meses;
for (int fila = 0; fila < tiposIntrs; ++fila)
{
    // Obtener el tipo de interés de la fila actual
    String sinteres = jtablaCabsFilas.getValueAt(fila, 0).toString();
    sinteres = sinteres.substring(0, sinteres.indexOf('%'));
    sinteres = sinteres.replace(',', '.');
    interes = Double.parseDouble(sinteres) / 100 / 12;
    // Calcular los pagos para este tipo de interés
    for (int columna = 0; columna < añosMeses; ++columna)
    {
        // Obtener la duración del préstamo
        colum = jtablaPrestamo.getColumnModel().getColumn(columna);
        String smeses = (String)colum.getHeaderValue();
        smeses = smeses.substring(0, smeses.indexOf(' '));
        meses = Integer.parseInt(smeses) * P;
        // Calcular la cantidad a pagar mensualmente
        if (interes == 0.0)
            pagoMensual = credito / meses;
        else
            pagoMensual = credito * (interes / (1 - (1 /
                (Math.pow(1.0+interes, (double)meses)))));
        // Ponerla en la tabla (se redondea a dos decimales)
        jtablaPrestamo.setValueAt(AlinDer("###,###,##0.00",
            pagoMensual), fila, columna);
    }
}
tablaPagos = true;
}

```

Se puede observar que para mostrar los datos ajustados a la derecha y redondeados a dos cifras decimales, el método anterior utiliza el método *AlinDer*. Por lo tanto, vamos a añadir a la clase *Prestamo* este método según se indica a continuación:

```

private StringBuffer AlinDer(String patrón, double dato)
{
    java.text.FieldPosition fp =
        new java.text.FieldPosition(java.text.NumberFormat.FRACTION_FIELD);
    java.text.DecimalFormat formato = new java.text.DecimalFormat(patrón);
    StringBuffer salida = new StringBuffer();
    formato.format(dato, salida, fp);
    for (int i = 0; i < (patrón.length() - fp.getEndIndex()); i++)
        salida.insert(0, ' ');
    return salida;
}

```

Cuando la rejilla visualiza la cantidad a pagar mensualmente para cada periodo y tipo de interés, el usuario puede seleccionar una de las celdas haciendo clic en ella, con el fin de visualizar la tabla de amortización para el crédito, en el periodo y tipo de interés a los que corresponde la mensualidad elegida. Como res-

puesta a este evento
ciaremos con la tabla

```

jtablaPrestamo
{
    public void m
    {
        jtablaPrest
    }
}

```

El método *jta*
que el usuario hizo
botón *Amortizaci*
tenga una cantidad
Añada este método

```

private void jt
{
    Object datoC

```

```

if (datoCeld
{
    // Pasar e
    StringBuffer
    for (int i
    {
        if (s.ch
        if (s.ch
    }
    // Convert
    pagoMensua
    jbtCalculo
}
}

```

Añada tamb
como propiedad

Para visuali
mente, el usuari
el método *jbtCa*

```

jbtCalculoAmor
{
    public void
    {
        jbtCalculo
    }
}

```

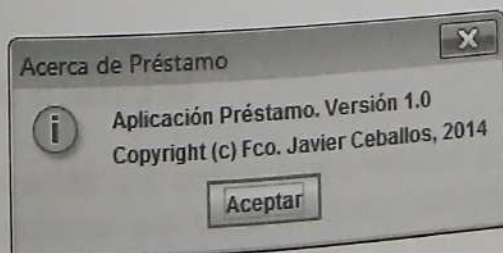
```

// Almacenar en la fila 0 las distintas cabeceras
string cab[] = {"Capital", "Intereses", "Capital pendiente",
               "Total intereses"};
for (columna = 0; columna < 4; ++columna)
{
    colum = jTablelaPrestamo.getColumnModel().getColumn(columna);
    colum.setHeaderValue(cab[columna]);
}

// Calcular y mostrar la tabla de amortización.
// La columna 0 de la tabla préstamo contiene el capital
// pendiente y la 1 contiene el interés mensual a pagar
double interesesMensuales = 0, creditoPendiente = credito;
double capitalMensualAmort = 0, totalIntereses = 0;
String formato = "###,###,##0.00";
for (int mes = 0; mes < meses; ++mes)
{
    // Cálculo de los interés a pagar en el mes actual
    interesesMensuales = creditoPendiente * interes;
    // Cálculo del capital en el mes actual
    capitalMensualAmort = pagoMensual - interesesMensuales;
    // Cálculo del capital pendiente de pagar
    creditoPendiente -= pagoMensual - interesesMensuales;
    // Cálculo de los intereses totales pagados
    totalIntereses += interesesMensuales;
    // Capital mensual amortizado
    jTablelaPrestamo.setValueAt (AlinDer(formato, capitalMensualAmort), mes, 0);
    // Intereses mensuales amortizados
    jTablelaPrestamo.setValueAt (AlinDer(formato, interesesMensuales), mes, 1);
    // Capital pendiente después de este pago
    jTablelaPrestamo.setValueAt (AlinDer(formato, creditoPendiente), mes, 2);
    // Total intereses abonados después de este pago
    jTablelaPrestamo.setValueAt (AlinDer(formato, totalIntereses), mes, 3);
}
jbtCalculoAmort.setEnabled(false);
tablaPagos = false;
}

```

Finalmente, vamos a implementar la orden *Acerca de...* del menú *Ayuda*. Esta orden visualizará una ventana que presentará los créditos de la aplicación:



Para ello, añade en el método *initComponents* de *Prestamo* un manejador de eventos de la clase **ActionListener** que vincule el objeto *jmlItemAcercaDe* con el método que responda a la acción "clic" realizada sobre él:


```
jmItemAcercaDe.addActionListener(new java.awt.event.ActionListener()
{
    public void actionPerformed(java.awt.event.ActionEvent evt)
    {
        jmItemAcercaDeActionPerformed(evt);
    }
});
```

A continuación añade a la clase *Prestamo* el método *jmItemAcercaDeActionPerformed* que mostrará un diálogo de la clase **JOptionPane** con una información análoga a la mostrada en la figura anterior.

EJERCICIOS PROPUESTOS

1. Modificar la aplicación *ArbolTfnos* para que, partiendo de un árbol vacío, permita:
 - a. Construir el árbol solicitando los datos de los nuevos teléfonos a través del teclado, para lo cual mostrará un diálogo.
 - b. Guardar los datos del árbol en un fichero cuando se cierre la aplicación.
 - c. Recuperar los datos desde el fichero cuando se inicie la aplicación.
 - d. Mostrar un menú que permita elegir los datos para iniciar el árbol entre varios ficheros.
2. Modificar la aplicación *Préstamo bancario* para que:
 - a. El texto de las cajas de texto que muestra la ventana marco quede seleccionado cuando reciban el foco, lo que facilitará su modificación.
 - b. Las cajas de texto que muestra la ventana marco solo admitan dígitos del 0 al 9 y el punto decimal.

```

new javax.swing.tree.DefaultTreeCellRenderer();
pintor.setClosedIcon(new javax.swing.ImageIcon(
    getClass().getResource("imagenes/telef-colgado.gif")));
pintor.setOpenIcon(new javax.swing.ImageIcon(
    getClass().getResource("imagenes/telef-descolgado.gif")));
pintor.setLeafIcon(new javax.swing.ImageIcon(
    getClass().getResource("imagenes/sonreir.gif")));
jTree1.setCellRenderer(pintor);

```

- Modificar el estilo de las líneas dibujadas entre los nodos. Esto puede hacerse asignando el valor adecuado a la propiedad **JTree.lineStyle**, invocando al método **putClientProperty** de **JTree**; el primer argumento de este método es la propiedad que se desea modificar del objeto, y el segundo, el nuevo valor que se desea asignar a la propiedad. Por ejemplo:

```
jTree1.putClientProperty("JTree.lineStyle", "Horizontal");
```

Otros posibles valores son "Angled" (valor por omisión) y "None".

EJERCICIOS RESUELTOS

Para ver con detalle la manera de utilizar una rejilla, vamos a desarrollar una aplicación que, a partir de los datos *crédito*, *tiempo* de amortización y tipo de *interés* al que se presta el mismo, visualice el pago mensual que debemos realizar para amortizar dicho crédito y la tabla de amortización mes a mes hasta la finalización del período del préstamo.

La aplicación deberá reunir fundamentalmente las siguientes características:

- Instrucciones para manipular la aplicación.
- El período de tiempo podrá venir dado en meses o en años.
- El pago mensual a calcular podrá visualizarse para más de un período y para más de un tipo de interés.

	10 años	11 años
2,00%	1.656,24	1.520,26
2,50%	1.696,86	1.561,13
3,00%	1.738,09	1.602,68

- La tabla de amortización incluirá por cada mensualidad su desglose en capital e intereses, el capital pendiente después de realizar ese pago y el total de los intereses pagados hasta ese momento.

	Capital	Intereses
1	325,07	450,00
2	325,88	449,19
3	326,70	448,37

Años	variable text mnemonic	jmlItemAños Años 'A'
Meses	variable text mnemonic	jmlItemMeses Meses 'M'
Ayuda	variable text mnemonic	jmnuAyuda Ayuda 'A'
Acerca de	variable text mnemonic	jmlItemAcercaDe Acerca de Préstamo... 'A'

El código siguiente crea la ventana marco y añade la barra de menús descrita en la tabla anterior:

```

/*
 * Prestamo.java
 */
package Cap05.Resueltos.Prestamo;

/**
 *
 * @author Fco. Javier Ceballos
 */
public class Prestamo extends javax.swing.JFrame
{
    /** Crear un nuevo formulario Prestamo */
    public Prestamo()
    {
        setTitle("Préstamo bancario");
        setSize(485, 385);
        initComponents();
    }

    private void initComponents()
    {
        jmlBarraDeMenus = new javax.swing.JMenuBar();
        jmnuOpciones = new javax.swing.JMenu();
        jmlItemInstruc = new javax.swing.JMenuItem();
        jmlSeparador1 = new javax.swing.JSeparator();
        jmnuSalir = new javax.swing.JMenu();
        jmnuPrestamoEn = new javax.swing.JMenu();
        jmlItemAños = new javax.swing.JMenuItem();
        jmlItemMeses = new javax.swing.JMenuItem();
        jmnuAyuda = new javax.swing.JMenu();
        jmlItemAcercaDe = new javax.swing.JMenuItem();

        getContentPane().setLayout(null);

        setResizable(false);
    }
}

```